Serial files

A serial file contains records that have not been organised in any defined order. A typical use of a serial file would be for a bank to record transactions involving customer accounts. A program would be running. Each time there was a withdrawal or a deposit the program would receive the details as data input and would record the data in a transaction file. In a serial file each new record is simply appended to the file so that the only ordering in the file is the time order of data entry.

Sequential files

A sequential file has records that are ordered. In the bank example, a sequential file could be used as a master file for an individual customer account. At regular periods of time, the transaction file would be read, and all affected customer account master files would be updated. In order to allow a sequential file to be ordered, there has to be a key field for which the values are unique and sequential but not necessarily consecutive. When a new record is to be added to a sequential file it would be possible to simply append the record, with the intention of sorting the file later. A more likely approach is for the file to be read sequentially and each record written to a new file. This is continued until the appropriate position for the new record is reached. The new record is then written to the new file before the remaining records in the old file are copied in.

Extension Question 16.01

Can you think of reasons why you might want to use binary files with variable-length records? How would you make sure a binary file with variable-length records would be read correctly?

Direct-access files

Direct-access files are sometimes referred to as 'random-access' files but, as with random-access memory, the randomness is only that the access can be to any record in the file without sequential reading of the file. Direct access can be achieved with a sequential file. A separate index file is created which has two fields per record. The first field has the key field value and the second field has a value for the position of this key field value in the main file.

The alternative is to use a hashing algorithm when a record is entered into the direct-access file.

One simple hashing algorithm is applicable if there is a numeric key field in each record. The algorithm chooses a suitable number and divides this number by the value in the key field. The remainder from this division then identifies the address in the file for storage of that record. The suitable number works best if it is a prime number of a similar size to the expected size of the file.

For simplicity this can be illustrated for 4-digit values in the key field where 1000 is used for the dividing number. The following represent three calculations:

0045/1000 gives remainder 45 for the address in the file

2005/1000 gives remainder 5 for the address in the file

3005/1000 gives remainder 5 for the address in the file

There are two facts apparent from these calculations. The first fact is that the addresses calculated do not have any order depending on the value in the key field. The second fact is that different key field values can produce the same remainder and therefore the same address in the file.

If the records do not have a suitable field with numeric digits, an alternative is to choose a field with some alphabetic characters. The ASCII code for each character can be looked up and the values then added. The sum is then used in the same way as described above, to calculate an address as the remainder from an integer division.

When the same address is calculated for different field values, it is usually referred to as a collision (the addresses are sometimes called synonyms). The best choice for a hashing algorithm is one that spreads the addresses most evenly and minimises the number of collisions. However, collisions cannot be avoided altogether so there has to be a defined method for dealing with collisons. There are a number of options, including the following:

- use a sequential search to look for a vacant address following the calculated one
- keep a number of overflow addresses at the end of the file
- have a linked list accessible from each address.

Question 16.01

Imagine the possible numeric values for a key field in a direct-access file are in the range of 1 to 30 but you want the file to have fewer than 30 file addresses.

You decide to test two examples of a modular division hashing algorithm. The first test uses 10 as the number for division, the second test uses 11.

a What are the two sets of addresses generated as remainders from the division for the key values 0 to 39 using 10 and 11?

b State one difference between the two sets of addresses.

c Is there any significant difference between the two sets of addresses?

d 11 is a prime number. Prime numbers are stated to give a better spread of use of the addresses in a file. Do you know when this is more likely to be true?

File access

Once a file organisation has been chosen and the data has been entered into a file, you need to consider how this data is to be accessed. For a serial file, the normal usage is to read the whole file record by record. If there was a need to search for a particular value in one of the fields, the only option would be to read the records from the beginning until the target record was found. If the data is stored in a sequential file and a particular value is needed, searching may have to be done in the same way. However, if the key field value is known for the record containing the wanted data, the process is faster because only key field values need to be read. For a direct-access file, the value in the key field is submitted to the hashing algorithm. The value is the same value that was used when entering the data originally and will provide the same value for the position in the file that was provided when the algorithm was used at the time of data input. This eliminates the need to read records from the beginning of the file. However, because of the collision problem some serial searching might be needed after the initial jump to the hashed position.

File access might also be needed to delete or edit data. For a sequential file the same method is used as when a new record was added. Records are copied from the old file to a new file until the record that needs to be deleted or edited is reached. Following deletion or editing all remaining records are copied to the new file.

For a direct-access file there is no need to create a new file. If a record needs editing it can be accessed directly and edited without disturbing any other content. However, if a record